

Worksheet — 2.2 Problem Solving & Programming

OCR A Level Computer Science (H446) — Component 02, Section 2.2 (2.2.1 Programming techniques · 2.2.2 Computational methods). All code = OCR Exam Reference Language.

Name: ____ Date: ____

Time: ~60 min. Check against the answer key at the end.

Section A — Skills practice

1. Key-term match

Write the letter of the correct definition next to each term.

Term	Answer		Definition
1. Recursion	_____	A	A subroutine that performs a task and returns a single value.
2. Base case	_____	B	The region of a program where a variable can be accessed.
3. Scope	_____	C	A subroutine that calls itself, with a stopping condition and a recursive case.
4. Function	_____	D	Passing a copy of an argument so the original is unaffected.
5. Procedure	_____	E	The condition that stops recursion — no further recursive call is made.
6. By value	_____	F	Passing the memory address, so the original can be changed.
7. By reference	_____	G	A stack of frames holding parameters, locals and return addresses for active calls.
8. Call stack	_____	H	A subroutine that performs a task but returns no value.

2. Predict the output — variable SCOPE (local vs global)

Study the code, then answer below. Remember: a **local** variable declared inside a subroutine is separate from a **global** of the same name and **shadows** it inside that subroutine.

```

global score = 10

procedure addPoints()
    score = 99          // refers to the GLOBAL score
endprocedure

procedure tryLocal()
    score = 5           // LOCAL score, declared inside here
    print(score)
endprocedure

print(score)
tryLocal()
addPoints()
print(score)

```

- What is printed by the **first** `print(score)` ? _____
- What is printed **inside** `tryLocal()` ? _____
- What is printed by the **last** `print(score)` ? _____
- Explain, in one sentence, why the assignment inside `tryLocal()` does **not** change what the final `print` shows.

3. Predict the output — pass BY VALUE vs BY REFERENCE

The same subroutine is called twice. Recall: **by value** passes a **copy** (original unchanged); **by reference** passes the **address** (original can be changed).

```

procedure increase(byVal n)
    n = n + 100
endprocedure

procedure boost(byRef n)
    n = n + 100
endprocedure

a = 1
increase(a)
print(a)

b = 1
boost(b)
print(b)

```

- What is printed for `a` ? _____

b) What is printed for `b`? _____

c) State the **consequence** (the marks-worthy point) that explains the difference — not just "a copy is made".

4. Recursion TRACE — complete the call stack

Here is a recursive factorial function:

```
function factorial(n)
  if n == 0 then
    return 1
  else
    return n * factorial(n - 1)
  endif
endfunction

print(factorial(4))
```

(a) Complete the table of stack frames as they are **pushed** (calls going down), and the **returned value** as each frame is **popped** (LIFO, going back up). One row is done for you.

Call (frame pushed)	Base case reached?	Expression returned	Value returned (on pop)
factorial(4)	No	4 * factorial(3)	---
factorial(3)	No	3 * factorial(2)	---
factorial(2)	No	-----	---
factorial(1)	No	-----	---
factorial(0)	Yes	return 1	1

(b) Final value printed by `print(factorial(4))`: _____

(c) What error occurs, and why, if the base case `if n == 0` were removed?

5. Write/complete a class in OCR pseudocode (with inheritance)

Complete the two classes below. `Vehicle` should store **make** and **numWheels** as **private** attributes set by a constructor, plus a method `describe()` that prints them. `Car` should **inherit** from `Vehicle`, add a private attribute **doors**, and **override** `describe()` to also print the number of doors.

```

class Vehicle
  private make
  private numWheels

  public procedure new(m, w)
    _____ = m
    _____ = w
  endprocedure

  public procedure describe()
    print("Make: " + make + ", wheels: " + numWheels)
  endprocedure
endclass

class Car _____ Vehicle    // keyword for inheritance
  private doors

  public procedure new(m, w, d)
    super.new(_____, _____)  // call the parent constructor
    doors = _____
  endprocedure

  public procedure describe()
    // override: reuse parent then add doors
    _____
    print("Doors: " + doors)
  endprocedure
endclass

myCar = new Car("Ford", 4, 5)
myCar.describe()

```

- a) Fill in every blank above.
- b) Name the OOP principle shown by keeping `make / numWheels` **private** and only changing them through methods. _____
- c) Name the OOP principle shown when `Car's describe()` replaces the inherited one. _____
-

6. Match the computational method to its use

Methods: **Backtracking** · **Data mining** · **Heuristics** · **Performance modelling** · **Pipelining** · **Visualisation**

Write the best-fit method next to each use.

- a) Splitting CPU instruction handling into fetch–decode–execute **stages** so different instructions are processed at once. → _____

- b) Searching a supermarket's huge loyalty-card records for hidden buying patterns and correlations. → _____
- c) A satnav using a **rule-of-thumb** (A*) to find a good-enough route quickly rather than the guaranteed-best one. → _____
- d) Solving a Sudoku by trying a value, and on a **dead end** returning to the last choice to try another. → _____
- e) Using a simulation to predict server load before the system is built. → _____
- f) Turning data into heat maps and charts to reveal trends and aid decisions. → _____
-

7. Challenge box

Challenge — convert iteration to recursion.

Here is an **iterative** countdown procedure:

```
procedure countdown(n) while n > 0 print(n) n = n - 1 endwhile print("Liftoff")
endprocedure
```

(i) Rewrite it as a **recursive** procedure `countdownR(n)` with the same output. Clearly mark your **base case** and your **recursive case**.

```
``` procedure countdownR(n)
```

---

---

---

---

```
endprocedure ```
```

(ii) State **one** advantage and **one** disadvantage of the recursive version compared with the iterative one.

---

---

## 8. Recursion trace II — sum of digits (examiner weak spot)

Examiners consistently report that recursion tracing is the weakest-answered topic in Component 02. Here is a second, different trace — this time the recursive call changes `n` with `DIV`, not subtraction.

```

function sumDigits(n)
 if n < 10 then
 return n
 else
 return (n MOD 10) + sumDigits(n DIV 10)
 endif
endfunction

print(sumDigits(4726))

```

(a) Complete the call-stack table. Frames are **pushed** going down the table; values are **returned** as each frame is **popped**, in LIFO order.

Call (frame pushed)	Base case reached?	Expression returned	Value returned (on pop)
sumDigits(4726)	No	6 + sumDigits(472)	__
sumDigits(_____)	No	_ + <i>sumDigits</i> (__)	__
sumDigits(_____)	No	_ + <i>sumDigits</i> (__)	__
sumDigits(_____)	Yes	return _____	__

(b) Final value printed by `print(sumDigits(4726))` : \_\_\_\_\_

(c) What is the **maximum number of `sumDigits` frames** on the call stack at the same time? \_\_\_\_\_

(d) Which call **finishes (returns) first**, and why does the LIFO behaviour of the stack make this so?

---

---

---

## 9. Find and fix the error — debugging OCR pseudocode

The function below is **intended** to return the largest value in the array `arr` (which always contains at least one number, and may contain negative numbers). It contains **three errors**: one **syntax** error and two **logic** errors.

```

01 function findMax(arr:array)
02 max = 0
03 for i = 0 to arr.length
04 if arr[i] > max then
05 max = arr[i]
06 next i
07 return max
08 endfunction

```

(a) Complete the table — identify each error, classify it, and give the fix.

Line	Syntax or logic?	What is wrong	Corrected line / fix
_____	_____	_____	_____
_____	_____	_____	_____
_____	_____	_____	_____

**(b)** Write out the fully corrected function.

```
function findMax(arr:array)
```

---



---



---



---



---



---

```
endfunction
```

**(c)** Give suitable **test data** that would reveal the error on line 02 even after the other two errors were fixed, and state the expected vs actual result.

---



---

**(d)** Which of the three errors would be reported by the translator **before the program runs**, and why are the others only found by **testing**?

---



---



---

## Section B — Exam practice (30 marks)

Answer all questions. Questions marked with an asterisk (\*) assess the quality of your extended response. All code must be written in OCR Exam Reference Language.

**Q1.** Describe the difference between a **local variable** and a **global variable**. [2] (AO1)

---



---



---

**Q2.** A subroutine `updateScores(list)` is passed a very large array.

(i) Explain why passing the array **by reference** is more efficient than passing it **by value**. [2] (AO2)

(ii) State **one** risk of passing the array by reference. **[1]** (AO2)

**Q3.** The following function is written in OCR Exam Reference Language.

```
function mystery(a, b)
 r = 0
 while b > 0
 if b MOD 2 == 1 then
 r = r + a
 endif
 a = a * 2
 b = b DIV 2
 endwhile
 return r
endfunction

print(mystery(5, 6))
```

(i) Complete the trace table for the call `mystery(5, 6)`. Record the values of `a`, `b` and `r` at the **end of each iteration** of the while loop. The starting values are given. **[3]** (AO2)

a	b	r	Output
5	6	0	

(ii) State the purpose of the function `mystery`. **[1]** (AO2)

**Q4.** A program compiles and runs but produces the wrong output. Identify **two** features of an IDE that would help the programmer locate the logic error at run time. **[2]** (AO1)



---

---

---

**Q5.** An array `scores` holds an unknown number of integers (zero-indexed; `scores.length` gives the number of elements). Write a function `countMatches(scores, target)` using **OCR Exam Reference Language** that returns the number of elements in `scores` equal to `target`. Your function must work for an array of **any** length. **[5]** (AO3)

---

---

---

---

---

---

---

---

**Q6.** A game contains characters of type **Wizard** and **Warrior**. Both types have a `name` and `health` attribute and a `move()` method; Wizards additionally have a `castSpell()` method. Explain how **inheritance** could be used when implementing these characters. **[3]** (AO2)

---

---

---

---

---

**Q7.** A route-planning app must find a route across a road network with millions of junctions. Explain why the app uses a **heuristic** approach rather than examining every possible route. **[2]** (AO1)

---

---

---

**Q8\*** A veterinary surgery is having software developed to store details of the animals it treats. The surgery treats dogs, cats and rabbits. All animals have a name, an owner and a date of last visit. Dogs additionally record whether they are microchipped; cats additionally record whether they are indoor-only.

Discuss how the developer could use **object-oriented programming** to implement this system. You should refer to **classes, inheritance, encapsulation and polymorphism** in your answer. **[9]** (AO3)

